Java Arrays

Introduction

- An array is a data structure that can store multiple values of the same type.
- Arrays in Java are fixed-size, meaning their length is defined when they are created and cannot be changed after that.
- Arrays allow for efficient access to elements using an index.

Accessing elements

Declaring an array

- int[] numbers;

Memory allocation

- numbers = new int[5]; // An array of 5 integers

Alternative declaration and initialization

- int[] numbers = {1, 2, 3, 4, 5};

// Accessing elements using the index

- int firstNumber = numbers[0]; // Access the first element

- // Modifying an element
- numbers[2] = 10; // Change the third element (index 2) to 10

Common Array operations

- int length = numbers.length;
- for (int i = 0; i < numbers.length; i++)

```
 {
  System.out.println(numbers[i]);
}
```

Multidimensional Array

- A multidimensional array is an array of arrays, allowing the storage of data in a table-like structure.

Declaration:

- int[][] arr = new int[3][4];

(2D Array with 3 rows and 4 columns).

- int[][][] arr = new int[3][4][5]; (3D Array).

- int[][] arr = {

{1, 2, 3},

{4, 5, 6},

{7, 8, 9}

};

- **Accessing Elements:**
  - arr[0][0] will access the element 1.
  - arr[2][1] will access the element 8.

Initialization of Multidimensional Arrays

- **Method 1: Static Initialization:**
  - int[][] arr = {{1, 2}, {3, 4}, {5, 6}};
- **Method 2: Dynamic Initialization:**
  - int[][] arr = new int[3][2];

Iterating Through Multidimensional Arrays

```
for(int i = 0; i < arr.length; i++)
{
for(int j = 0; j < arr[j].length; j++)
{
System.out.print(arr[i][j] + " ");
}
System.out.println();
}
```

This program adds two matrices of the same dimensions.

```
public class MatrixAddition {
   public static void main(String[] args) {
      int[][] matrix1 = {
          {1, 2, 3},
          {4, 5, 6},
         {7, 8, 9}
       };

     int[][] matrix2 = {
          {9, 8, 7},
          {6, 5, 4},
          {3, 2, 1}
       };
```

```
int[][] result = new int[3][3]; // Resultant matrix

        // Adding matrices
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                result[i][j] = matrix1[i][j] + matrix2[i][j];
            }
        }

        // Displaying result
        System.out.println("Matrix after addition:");
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(result[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

- Transpose a 2D array
- Sum of all elements of a 2D array
- Maximum element of a 2D array
- Row and Column sum of a matrix
- Check two matrices are equal
- Find the row with maximum sum
- Check whether the matrix is symmetric or not

STRINGS

- A **String** is a sequence of characters.
- **String class** in Java is used to create and manipulate strings.
- **String** is immutable in Java.
  - A String object is created in two ways:
    - **String Literal**: String str = "Hello";
    - **Using new keyword**: String str = new String("Hello");

- **Important Methods:**
  - length(): Returns the length of the string.
  - charAt(index): Returns the character at the specified index.
  - concat(): Concatenates two strings.
  - substring(): Extracts a substring from the string.
  - equals(): Compares two strings for equality.
  - toLowerCase(), toUpperCase():

- String str1 = "Hello";
- String str2 = " World";
- // Concatenation
- String str3 = str1.concat(str2);
  System.out.println(str3); // Output: Hello World
- // Substring
- String subStr = str3.substring(6, 11);
  System.out.println(subStr); //World

StringBuffer

- Unlike String, **StringBuffer** is **mutable**.
- It allows modification of the string without creating a new object.
- StringBuffer objects can be modified efficiently without creating new objects.

Important Methods

- append(): Adds text to the end of the current string buffer.
- insert(): Inserts text at a specific index.
- delete(): Removes text between specified indices.
- reverse(): Reverses the content of the string buffer.
- capacity(): Returns the current capacity of the string buffer.
  - Default capacity = **16 characters**

```
StringBuffer sb = new StringBuffer("Hello");
// Append text
sb.append(" World");
System.out.println(sb);  // Output: Hello World

// Insert text at a specific position
sb.insert(5, ",");
System.out.println(sb);  // Output: Hello, World

// Reverse the string
sb.reverse();
System.out.println(sb);  // Output: dlroW ,olleH
```

```
StringBuffer message = new StringBuffer("Hello");
message.append(" Students");// Hello Students
message.insert(5, " Dear");// Hello Dear Students
message.replace(6, 10, "Friends");
// Hello Friends Students
message.delete(5, 6);
//HelloFriends Students
```

Palindrome checker

```
import java.util.*;

class PalindromeCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String input = sc.nextLine();

        StringBuffer sb = new StringBuffer(input);
        String reversed = sb.reverse().toString();

        if (input.equals(reversed))
            System.out.println("Palindrome");
        else
            System.out.println("Not a Palindrome");
    }
}
```

String vs StringBuffer

Feature	**String**	**StringBuffer**
Mutability	Immutable	Mutable
Performance	Slower with modifications	Faster with modifications
Thread Safety	Not thread-safe	Thread-safe
Usage	Suitable for constant values	Suitable for frequent modifications

Program to search a name

```
import java.util.Scanner;

public class NameSearch {
    public static void main(String[] args) {
        // Array of names
        String[] names = {"Alice", "Bob", "Charlie", "David", "Emma"};

        // Scanner for user input
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a name to search: ");
        String searchName = scanner.nextLine();

        // Convert search name to lowercase for case-insensitive search
        boolean found = false;
        for (int i = 0; i < names.length; i++) {
            if (names[i].equalsIgnoreCase(searchName)) {
                System.out.println("Name found at index: " + i);
                found = true;
                break;
            }
        }
```